The Ultimate Guide to

Exploratory Data Analysis (EDA)

A practical, professional-grade tutorial covering core concepts, statistical foundations, and advanced visualization techniques for EDA.

Exploratory Data Analysis (EDA) is a mandatory, foundational step before deploying any machine learning models. It requires focus and patience. Before extracting meaningful insights, you will often spend significant time actively exploring your data through statistical summaries and visualization libraries.

When approaching EDA, adopt the professional formula:

Customize → Itemize → Optimize

- **Customize** your charts for specific needs (avoid relying solely on basic plots).
- **Itemize** important details on your charts so they speak for themselves.
- **Optimize** your workflow to make the EDA process faster and cleaner.

1. Why Do EDA?

Performing EDA serves several critical purposes in the data science lifecycle:

- **Model Building:** Understanding the data shapes the architecture of your predictive models.
- **Analysis and Reporting:** Extracting actionable business insights.
- **Validating Assumptions:** Ensuring the data behaves as expected before applying complex algorithms.
- **Handling Missing Values:** Identifying gaps in data and determining imputation strategies.
- **Feature Engineering:** Discovering relationships that allow you to create new, powerful predictive columns.
- **Detecting Outliers:** Spotting anomalies that could skew model performance.

2. Understanding Column Types

Before analyzing data, you must classify your variables. This dictates the statistical methods and visualizations you will use.

Data Type	Description & Examples
Numerical	Continuous or discrete numbers. Examples: Age, Fare, PassengerId
Categorical	Distinct categories or groups. Examples: Survived, Pclass, Sex, SibSp, Parch, Embarked

Data Type	Description & Examples
Mixed	A combination of text and numbers. Examples: Name, Ticket, Cabin

3. Univariate Analysis

Univariate analysis focuses on analyzing each feature independently. The goal is to examine its shape, central tendency, dispersion, and identify potential issues like outliers, skewness, and missing values.

The Shape of Data

The shape of a data distribution refers to its overall pattern on a graph. Common shapes include:

- **Normal Distribution:** Symmetrical and bell-shaped. Mean, median, and mode are equal.
- **Skewed Distribution:** Asymmetrical, with one tail longer than the other. Can be positively (right) or negatively (left) skewed.
- **Bimodal Distribution:** A distribution containing two distinct peaks (modes).
- **Uniform Distribution:** All values have an equal (or near-equal) chance of occurring.

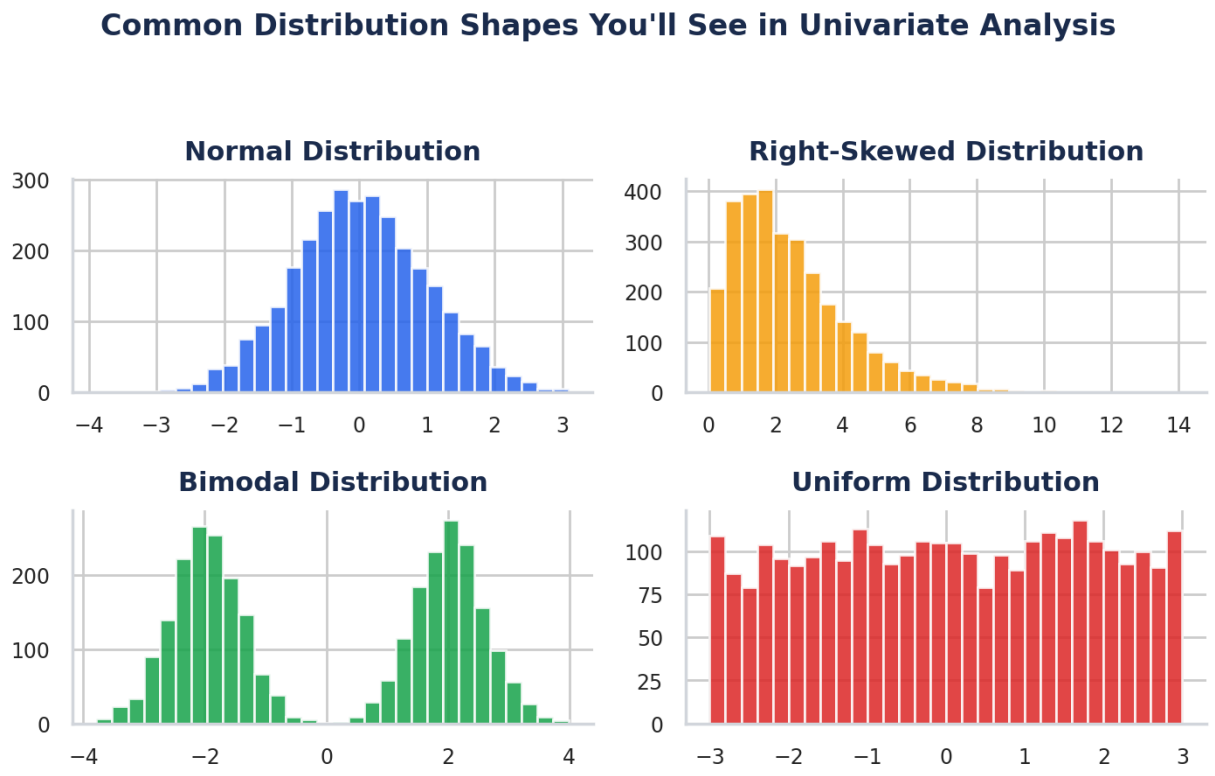


Fig 1. Simulated data showing the four common distribution shapes side by side.

Measures of Dispersion

Dispersion describes the spread or variability of your data relative to its central tendency.

- **Range:** The difference between the maximum and minimum values.
- **Variance:** The average of the squared deviations from the mean.

- **Standard Deviation:** The square root of the variance (expressed in original units).
- **Interquartile Range (IQR):** The range between the 25th percentile (Q1) and 75th percentile (Q3). Excellent for identifying outliers.

A. Analyzing Numerical Columns

Step-by-step approach:

- **Descriptive Statistics:** Compute mean, median, mode, standard deviation, range, and quartiles.
- **Visualizations:** Plot Histograms, Box plots, and Density (KDE) plots.
- **Identify Outliers:** Use box plots to find anomalies. Determine if they are errors or legitimate data points.
- **Check Skewness:** If highly skewed, consider log transformations or robust statistical methods.
- **Conclude:** Summarize findings to guide feature engineering.

> ***Example Notes (Titanic Dataset):***
>
> **Age:** Almost normally distributed. Approximately 20% of values are missing. Contains some outliers.
>
> **Fare:** Highly (positively) skewed. The column appears to represent group fares rather than individual fares. **Actionable Insight:** We need to engineer a new 'individual_fare' column.

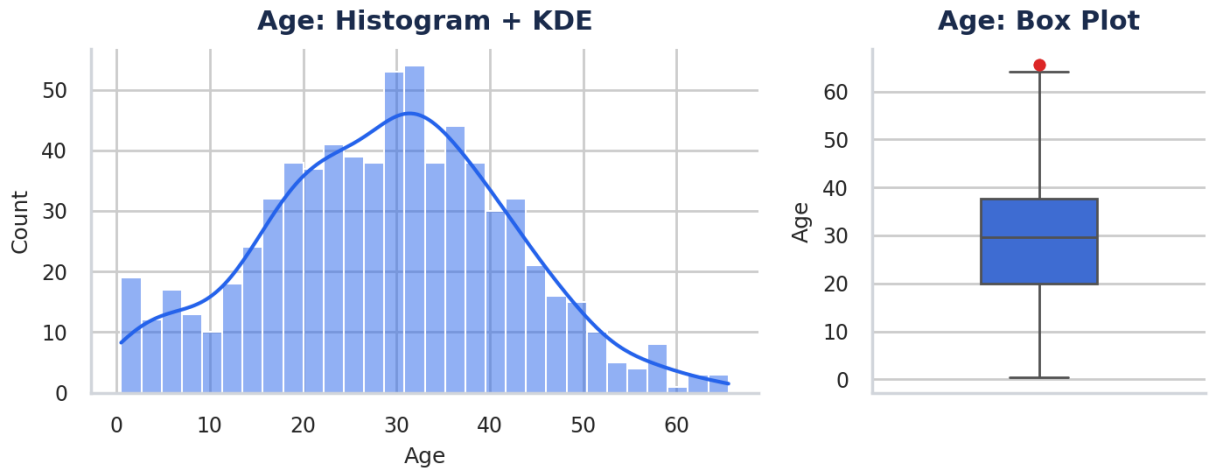


Fig 2. Age is roughly bell-shaped with a mild right tail; the box plot flags a handful of elderly-passenger outliers on the high end.

Outlier Detection Deep-Dive: The IQR Rule

The most common rule of thumb for flagging outliers in a numerical column uses the Interquartile Range (IQR). Any point outside the fence below is treated as a candidate outlier worth investigating — not automatically deleting.

$$IQR = Q3 - Q1$$

$$\text{Lower bound} = Q1 - 1.5 \times IQR \quad | \quad \text{Upper bound} = Q3 + 1.5 \times IQR$$

```
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df['Fare'] < lower_bound) | (df['Fare'] > upper_bound)]
print(f"{len(outliers)} outliers found, upper bound = {upper_bound:.2f}")
```

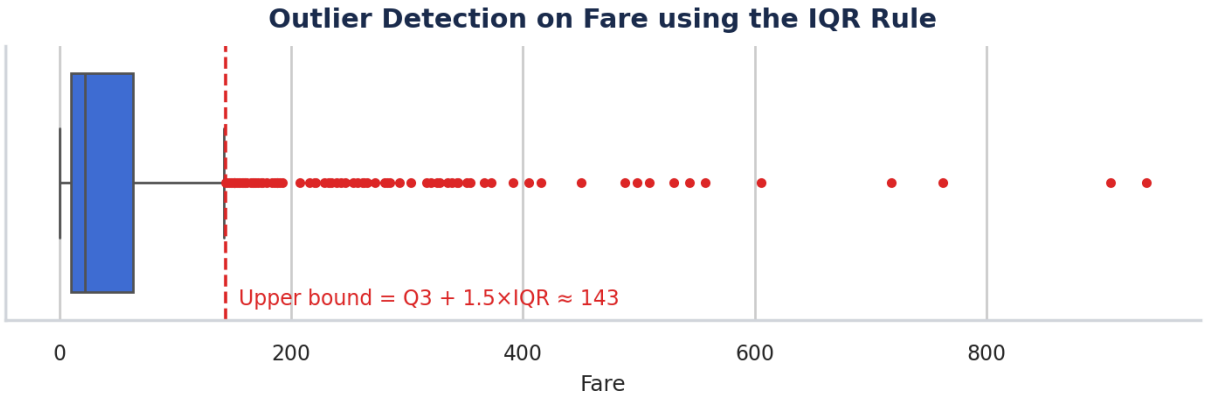


Fig 3. Fare has a long right tail; the IQR rule places the upper fence around ~143, so most first-class group fares above that are flagged as candidate outliers, not necessarily errors.

> ***Alternative: The Z-Score Method***
>
> For roughly normal columns, flag any value where |z| > 3, where z = (x – mean) / std. This works well for Age (which is close to normal) but performs poorly on Fare, since Fare's skew distorts the mean and standard deviation — another reason to inspect the shape first.

B. Analyzing Categorical Columns

Step-by-step approach:

- **Descriptive Statistics:** Compute the frequency distribution (counts) of each category.
- **Visualizations:** Plot Count plots and Pie charts.
- **Missing Values:** Determine if missing categories should be imputed (e.g., using the mode) or treated as a separate "Unknown" category.
- **Conclude:** Summarize findings.

> ***Example Notes (Titanic Dataset):***
>
> **Survived / Family Size:** Insights suggest that Parch (Parents/Children) and SibSp (Siblings/Spouses) can be merged to engineer a family_size column, and subsequently, a binary is_alone column.

Pro-Tip 4: Visualize Missing Data First

Before deciding how to impute or drop missing values, get a full picture of where the gaps are. A quick horizontal bar chart of missing-percentage per column is faster to scan than scrolling through df.isnull().sum() output, and it makes it obvious which columns are lightly affected versus which ones (like Cabin) are missing too much to impute reliably.

```
missing_pct = df.isnull().mean().sort_values() * 100

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 3))
ax.barh(missing_pct.index, missing_pct.values)
ax.set_xlabel("% Missing")
ax.set_title("Missing Data by Column")
plt.show()
```

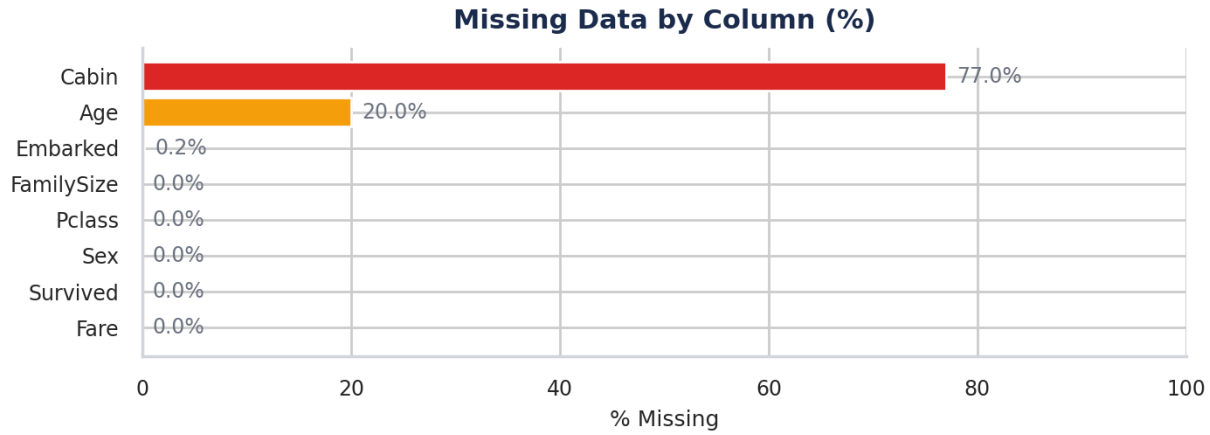


Fig 4. Cabin is missing in the large majority of rows (drop or bucket into 'has_cabin' / 'unknown'), while Age is missing enough to justify model-based imputation rather than dropping rows.

> ***Decision Rule of Thumb:***
>
> < 5% missing → impute with mean/median/mode. 5–40% missing → impute with a model or a group-wise median (e.g. median Age per Pclass & Sex). > 60% missing → usually better to drop the column or turn it into a binary 'is_missing' flag rather than force an imputation.

Pro-Tip 5: Fix Skewness with a Log Transform

Highly skewed numerical columns (like Fare) can distort correlation coefficients, distance-based models, and even simple bar/box comparisons. A quick log1p transform (log(1 + x), which safely handles zeros) often pulls the distribution back toward something closer to normal.

```
import numpy as np

print("Raw skew:", df['Fare'].skew())          # 4.33  (highly skewed)
df['Fare_log'] = np.log1p(df['Fare'])
print("Log skew:", df['Fare_log'].skew())       # 0.31  (near-normal)
```

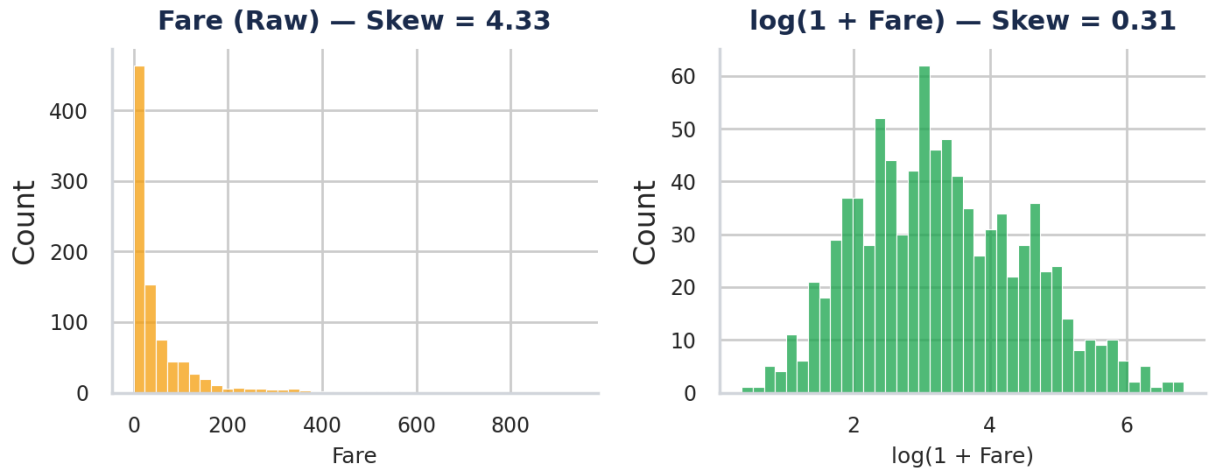


Fig 5. The raw Fare column (skew ≈ 4.3) becomes far closer to symmetric after a log1p transform (skew ≈ 0.3) — much friendlier input for linear models and correlation analysis.

4. Bivariate & Multivariate Analysis

Bivariate analysis explores the relationship between two variables, while multivariate looks at more.

Relationship Type	Recommended Visualizations	Analytical Goal
Numerical vs. Numerical	Scatterplots, 2D Histograms, 2D KDE plots.	Check linear relationships via correlation coefficients.
Numerical vs. Categorical	Bar plots, Box plots, Violin plots, KDE plots grouped by category.	Compare the numerical distribution across different categorical groups.
Categorical vs. Categorical	Heatmaps, Stacked bar plots, Tree maps.	Create cross-tabulations or contingency tables to see group overlaps.

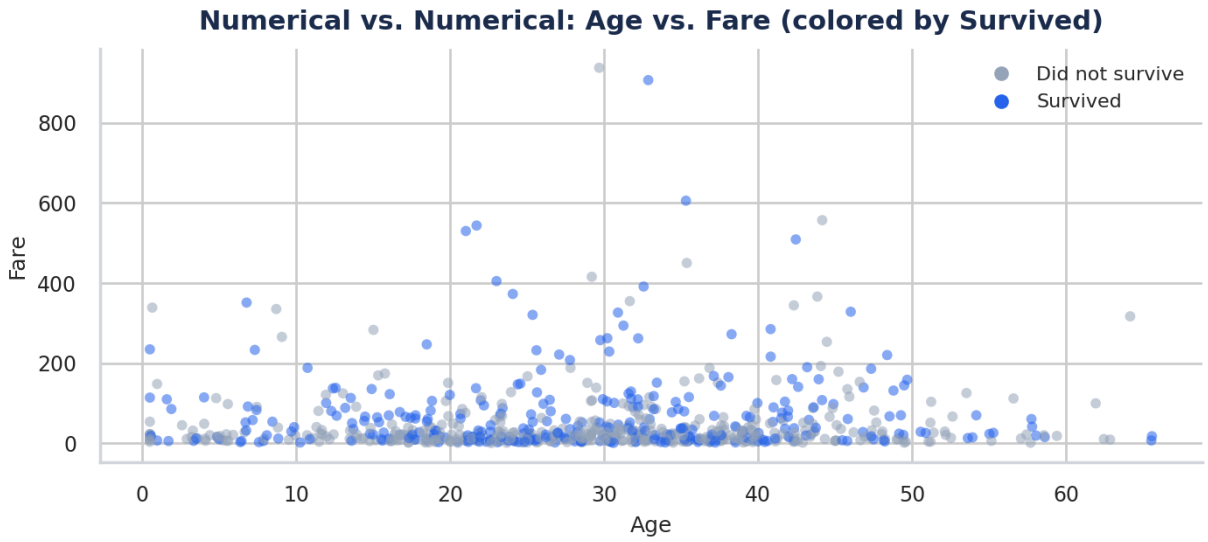


Fig 6. Numerical vs. numerical: Age vs. Fare, colored by survival. First-class passengers (high fare) survived at much higher rates regardless of age.

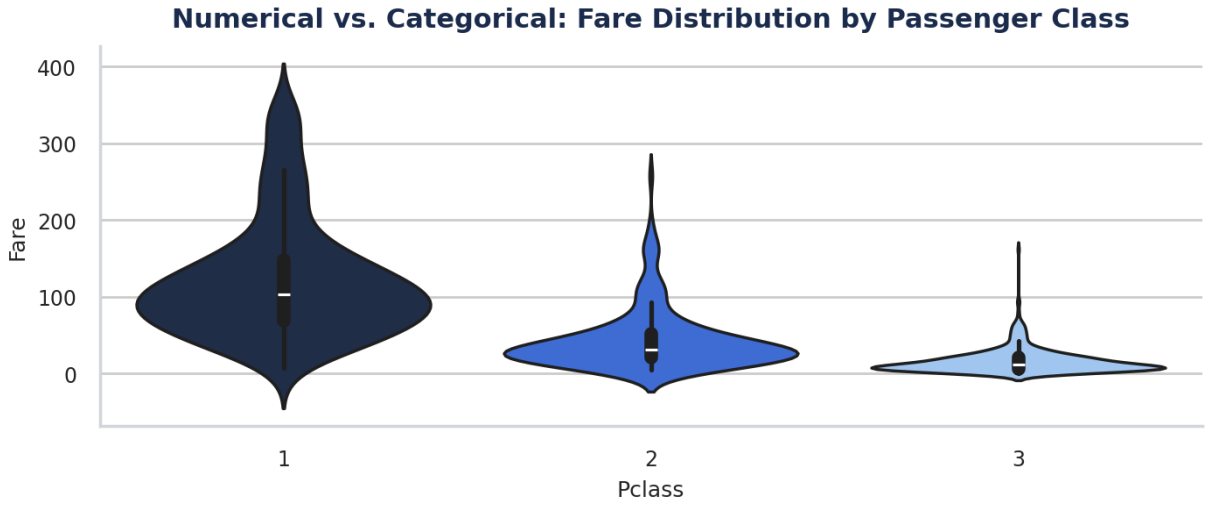


Fig 7. Numerical vs. categorical: Fare distribution split by Pclass. Class 1 fares are both higher and far more spread out than classes 2 and 3.

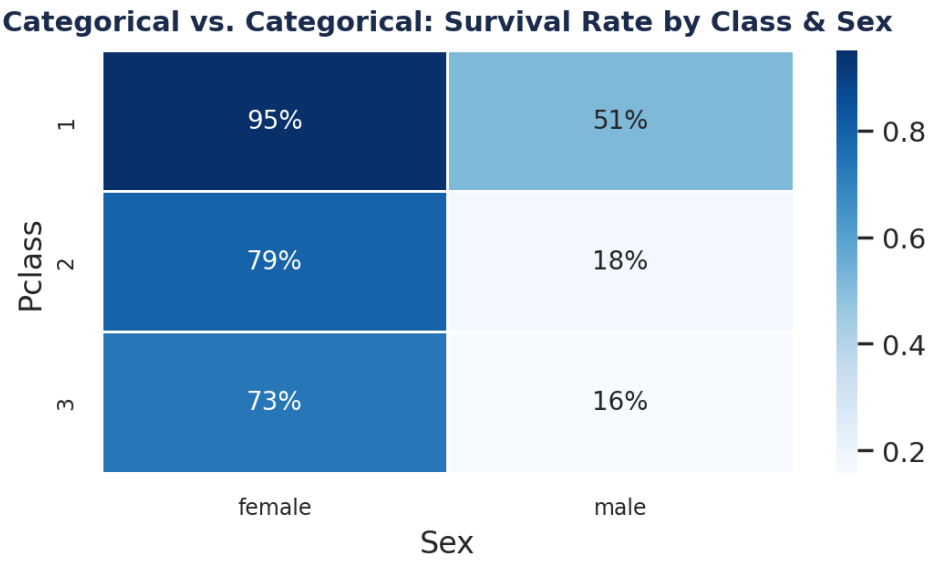


Fig 8. Categorical vs. categorical: survival rate by class and sex. Sex is the single strongest predictor here, with class amplifying the effect within each sex.

Pro-Tip 1: Don't Be Afraid to Customize Non-Trivial Charts

Standard heatmaps (like a basic correlation matrix) only show numbers and can be difficult to interpret quickly. A standard scatterplot matrix (like sns.pairplot) is better for visualizing distributions and outliers but creates a lot of duplicate graphical information.

The Solution: Combine the best of both worlds into a custom summary matrix using seaborn.PairGrid.

- **Lower Diagonal:** Scatter plots to view data spread and outliers.
- **Diagonal:** Histograms/KDEs to view individual feature distributions.
- **Upper Diagonal:** Scaled correlation dots (larger/darker circles indicate stronger relationships).

```
import seaborn as sns
import matplotlib.pyplot as plt

def correlation_dots(*args, **kwargs):
    corr_r = args[0].corr(args[1], 'pearson')
    ax = plt.gca()
    ax.set_axis_off()
    marker_size = abs(corr_r) * 3000
    ax.scatter([.5], [.5], marker_size, [corr_r], alpha=0.5,
               cmap='Blues', vmin=-1, vmax=1, transform=ax.transAxes)

# Assuming 'data' is your DataFrame
fig = sns.PairGrid(data, aspect=1.4, diag_sharey=False)
fig.map_lower(sns.regplot)
fig.map_diag(sns.histplot)
fig.map_upper(correlation_dots)
plt.show()
```

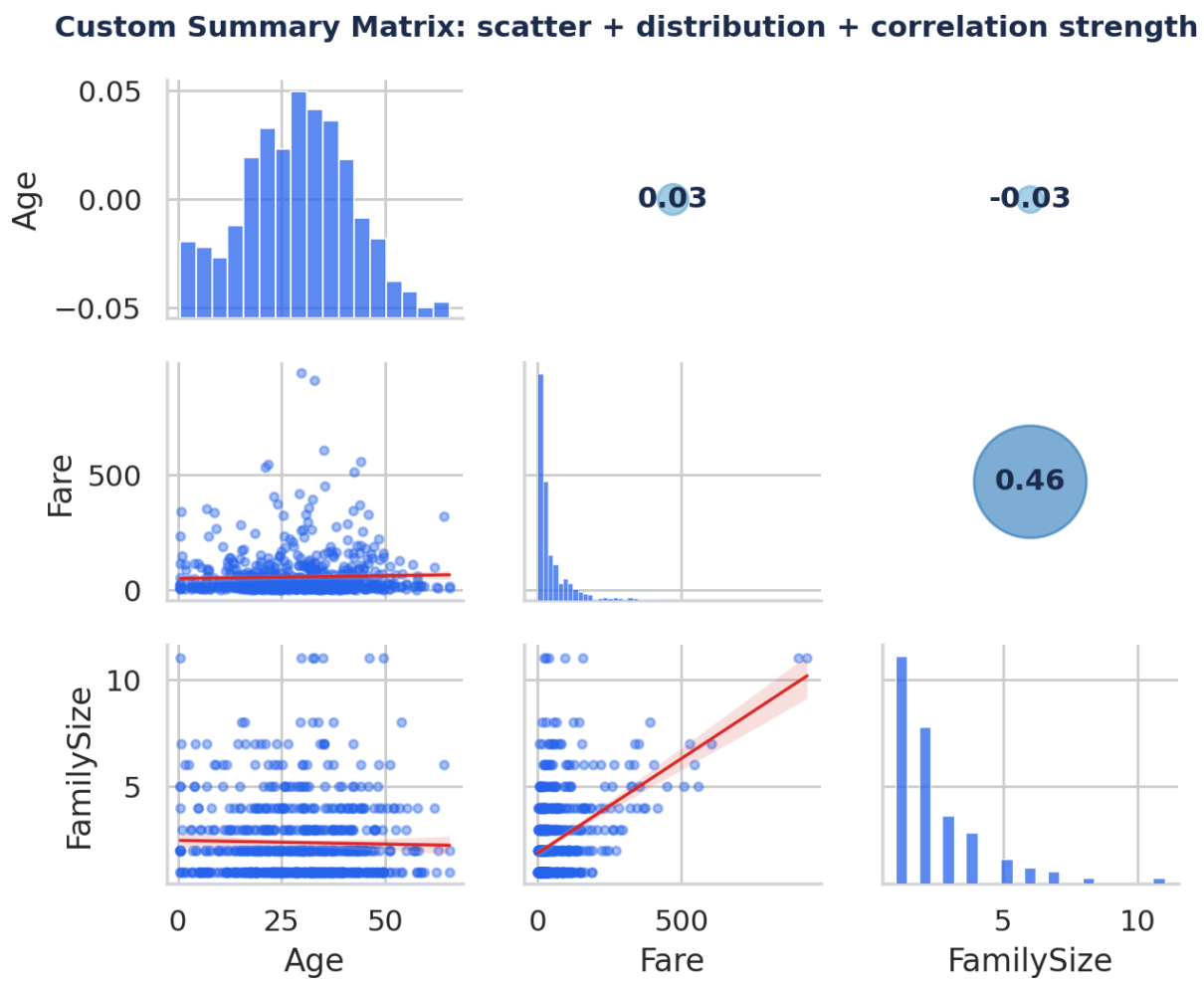


Fig 9. The custom PairGrid in action on Age, Fare, and FamilySize: scatter + trend line below the diagonal, distributions on the diagonal, and correlation strength as scaled dots above — three views of the same variables without repeating information.

5. Advanced Visualization Pro-Tips

Pro-Tip 2: Itemize Your Charts ("Make Them Talkative")

When presenting EDA to stakeholders, a “silent” chart with just axes and data points often requires you to be in the room to explain it. Make your charts “talkative” by utilizing the full power of your visualization library to add context.

Add vertical/horizontal lines, text annotations, and arrows to highlight critical zones or extreme outliers directly on the graph.

```
plt.scatter(data['Speed'], data['Power'], label='actual')
plt.scatter(data['Speed'], data['Power_curve'], label='theoretical')

# Adding context lines and text
plt.vlines(x=3.05, ymin=10, ymax=350, lw=3, color='black')
plt.text(1.1, 355, r"cut-in", fontsize=15)

# Highlighting an outlier with an arrow
plt.annotate('outlier!', xy=(18.4,1805), xytext=(21.5,2050),
             arrowprops={'color':'red'})
```

```
plt.legend(loc='best')
plt.show()
```

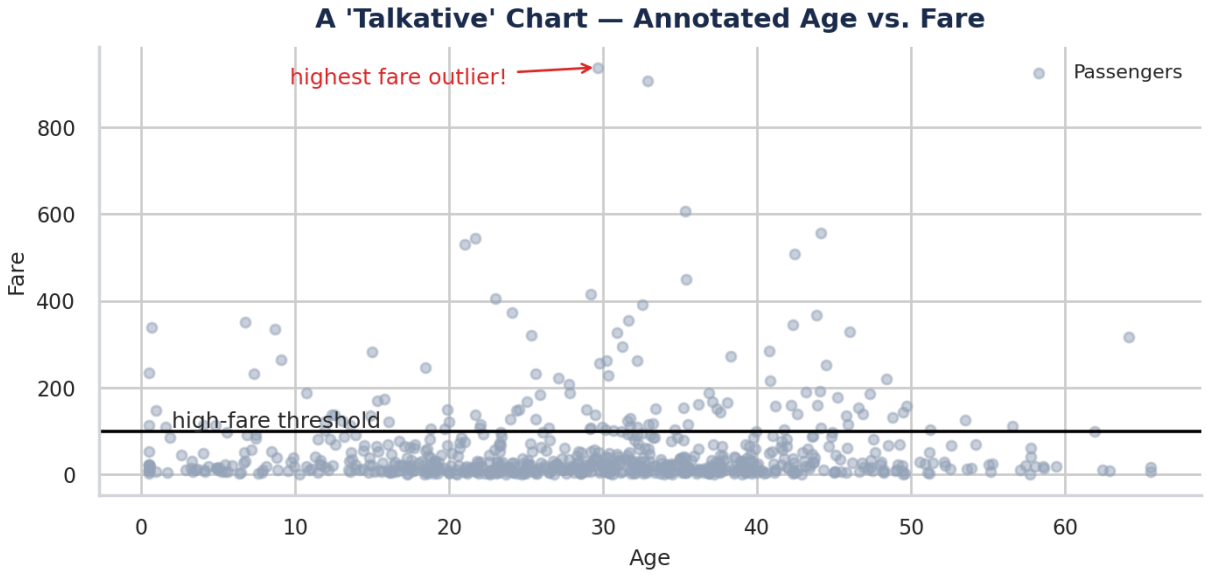


Fig 10. The same Age-vs-Fare scatter as before, but made 'talkative': a threshold line marks the high-fare zone and an annotation calls out the single highest-fare passenger directly on the chart, so it explains itself without a live narrator.

Pro-Tip 3: Optimize Your Workflow (Find Faster Ways to Build)

Sometimes, fighting with matplotlib subplots (like combining a standard line chart with a specialized polar windrose chart) leads to messy formatting and uneven whitespace.

Instead of forcing complex subplots in matplotlib, generate your distinct charts, save them as images, and use the Python Imaging Library (PIL) to quickly stack them perfectly.

Code Snippet for Stacking Images with PIL:

```
import numpy as np
import PIL

# Load images
list_im = ['chart_1.png', 'chart_2.png']
imgs = [PIL.Image.open(i) for i in list_im]

# Find the smallest image shape to resize the others
min_shape = sorted([(np.sum(i.size), i.size) for i in imgs])[0][1]

# Stack vertically (use np.hstack for horizontal)
images_comb = np.vstack((np.asarray(i.resize(min_shape)) for i in imgs))
images_comb = PIL.Image.fromarray(images_comb)

# Save the perfectly merged infographic
images_comb.save('merged_charts.png', dpi=(600,600))
```

6. Common Pitfalls to Avoid

- **Dropping missing rows too early:** you may be discarding a pattern (e.g. missing Cabin often correlates with lower class) that is itself predictive.
- **Trusting the mean on skewed data:** for columns like Fare, the median and IQR describe the data far better than the mean and standard deviation.
- **Treating ordinal data as purely categorical or purely numerical:** Pclass has an inherent order (1 > 2 > 3 in status) — encode and visualize it accordingly rather than as unordered categories.
- **Reading correlation as causation:** a strong correlation between Fare and Survived doesn't mean paying more caused survival — both are downstream of Pclass and cabin location.
- **Only looking at aggregate statistics:** always pair a summary statistic (mean, skew, correlation) with a picture; distributions with identical means can look completely different.

7. Quick-Reference EDA Checklist

Step	What to Do
1. Inspect	df.shape, df.info(), df.head(), df.describe() on every new dataset.
2. Classify columns	Split into numerical, categorical, and mixed-type columns.
3. Check missingness	Plot % missing per column; decide impute / flag / drop per column.
4. Univariate pass	Histogram + box plot for numerics; count plot for categoricals.
5. Outlier check	Apply the IQR rule (or Z-score for near-normal columns).
6. Transform if needed	Log-transform highly skewed numeric columns before modeling.
7. Bivariate pass	Scatter/violin/heatmap against the target variable for every feature.
8. Engineer features	Combine or bucket raw columns based on the patterns you found.
9. Document insights	Write down the 'why' behind each transform, not just the 'what'.